

Ejercicios Colecciones Kotlin

[Descargar estos ejercicios](#)

Ejercicio 1

Teniendo en cuenta las colecciones vistas en el tema, indica qué tipo de colección sería el más adecuado para cada caso y crea una variable inicializada con un ejemplo. Decide si debe ser **var** o **val** según la mutabilidad que necesites.

1. Lista con los nombres de los alumnos de una clase para mostrarlos en pantalla, los alumnos no van a cambiar durante el curso.
2. Colección para guardar la nota final de cada alumno de un grupo, de forma que cada alumno tenga asociado un valor numérico.
3. Conjunto con los códigos de los ordenadores disponibles en un aula, sin que se repitan.
4. Lista mutable con los ejercicios de programación que debe entregar un estudiante durante el curso, pudiendo añadir y quitar tareas.
5. Relación entre cada día de la semana y la asignatura que se imparte a esa hora.
6. Colección para guardar los idiomas que domina una persona, sin repetir ninguno.
7. Lista mutable con los precios de los productos de un supermercado que cambian cada semana.
8. Colección para guardar el nombre del usuario y su contraseña en una aplicación, donde el usuario será la clave y la contraseña podrá modificarse.
9. Colección para almacenar el menú de cada día de la semana en el comedor del instituto, usando un tipo **Pair** o un **Map**.
10. Conjunto de palabras clave que describen un proyecto de programación, sin duplicados.

Ejercicio 2

Vamos a crear la parte de una aplicación que gestiona el inventario de una pequeña tienda. Para ello crearemos un tipo **object GestionInventario** con un campo privado de tipo **MutableMap<String, Int>** donde la clave será el código del producto y el valor será la cantidad disponible en stock.

Además, crearemos una excepción **InventarioException** para controlar errores y funciones para añadir, modificar y eliminar productos del almacén.

Antes de cerrar el programa, se mostrará el contenido del map para comprobar que la gestión se ha realizado correctamente.

El programa principal podría mostrar un menú como este:

1. Añadir producto
2. Modificar stock
3. Eliminar producto
4. Salir



Aviso

Al añadir y eliminar se deberá comprobar que el código no esté vacío. Al modificar el stock, el producto debe existir y se deberá validar que la cantidad introducida sea mayor o igual a 0.

Ejercicio 3

Vamos a crear una pequeña aplicación para gestionar una colección de películas que están en cartelera. Queremos almacenar información sobre cada película y luego poder buscar y ordenar la colección de forma cómoda.

Crea un **data class Pelicula** con los siguientes campos:

- título: String
- género: String
- duración: Int
- director: String

Usa una colección adecuada para guardar todas las películas y aplica funciones de **kotlin.collections** como `filter`, `sortedBy`, `groupBy`, `count`, `maxByOrNull`, `partition` o `associateBy` para resolver estas tareas:

1. Muestra todas las películas con sus datos.
2. Busca las películas cuyo título contenga una palabra introducida por teclado, ignorando mayúsculas/minúsculas.
3. Muestra solo las películas de un género concreto.
4. Obtén la película más larga de la colección.
5. Separa las películas en dos grupos: las de 120 minutos o más y las más cortas.

Puedes probar el programa con esta lista inicial:

```
val peliculas = listOf(  
    Pelicula("Dune", "Ciencia Ficción", 155, "Denis Villeneuve"),  
    Pelicula("El viaje de Chihiro", "Animación", 125, "Hayao Miyazaki"),  
    Pelicula("La la land", "Musical", 128, "Damien Chazelle"),  
    Pelicula("Mad Max: Fury Road", "Acción", 120, "George Miller"),  
    Pelicula("Parasite", "Thriller", 132, "Bong Joon-ho"),  
    Pelicula("Pride and Prejudice", "Romance", 128, "Joe Wright"),  
    Pelicula("Shrek", "Animación", 90, "Andrew Adamson")  
)
```

Crea un menú con las distintas opciones para poder ejecutar la app hasta no seleccionar la opción de salir.